

ARDUINO

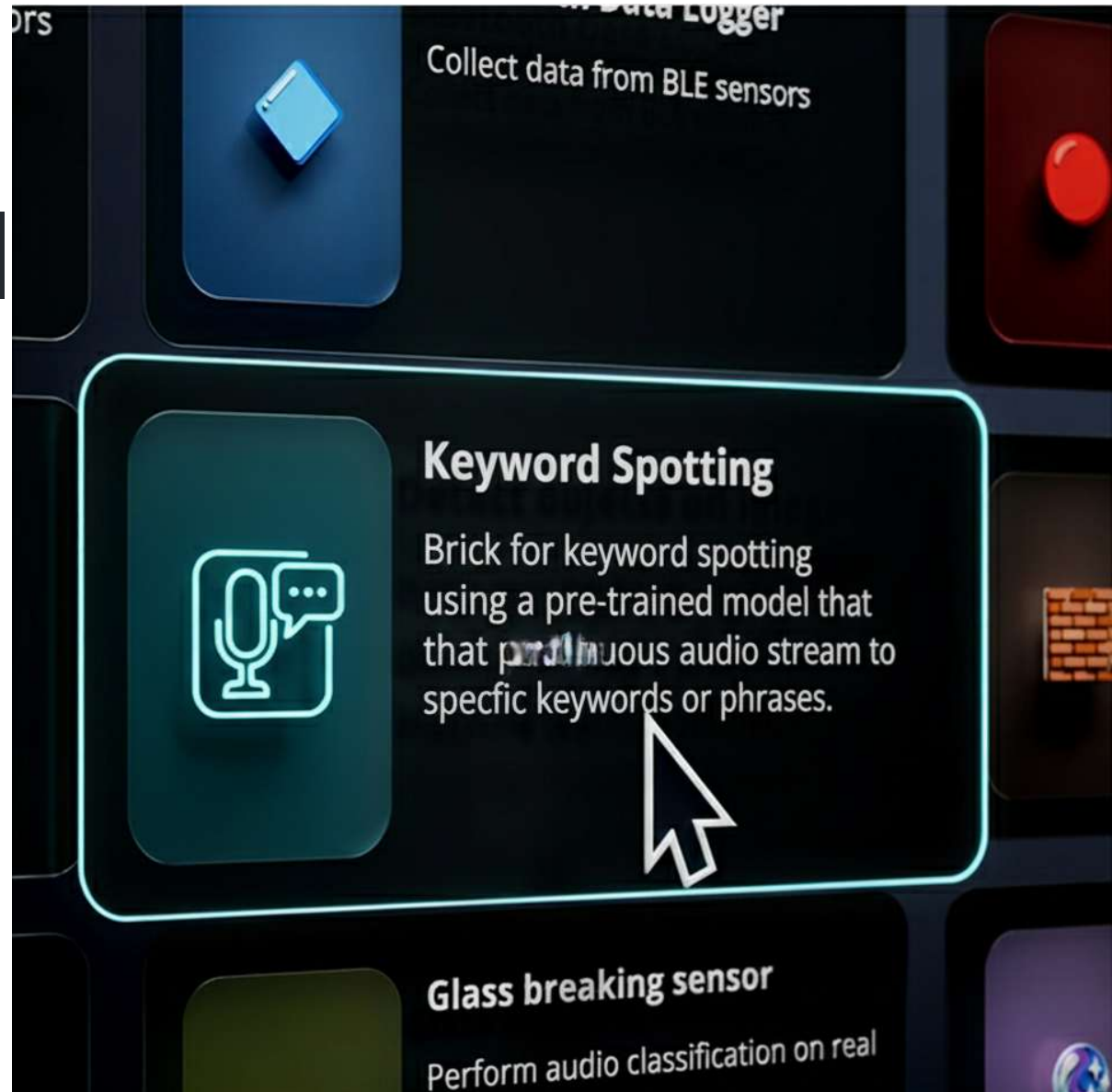
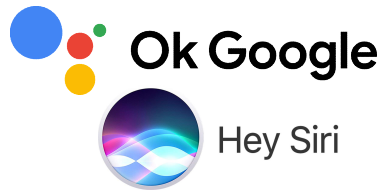
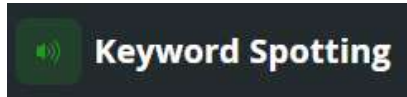
UNO

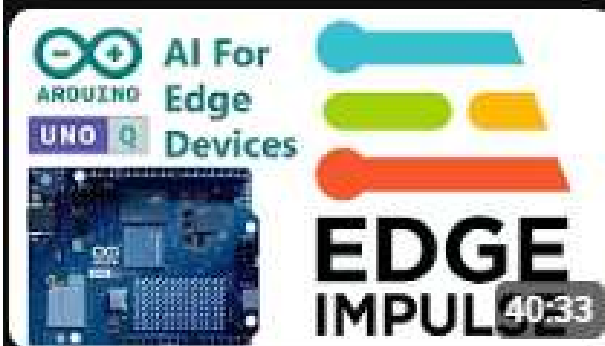
Q

AI For Edge Devices

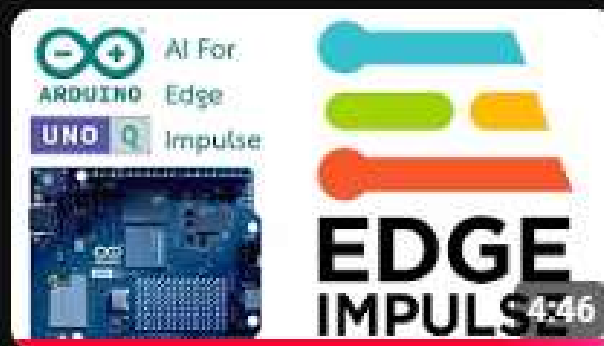


EDGE IMPULSE

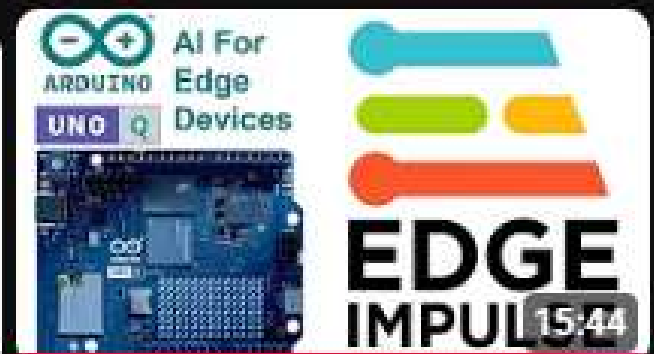




Arduino UNO Q (Ep.36) การ
เทรนโมเดล AI ให้ใช้กับงาน...



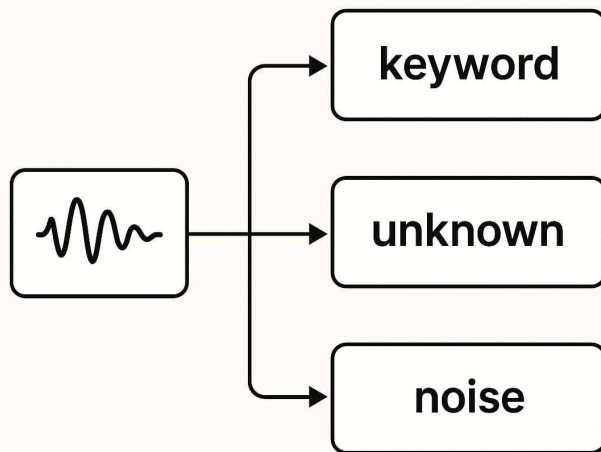
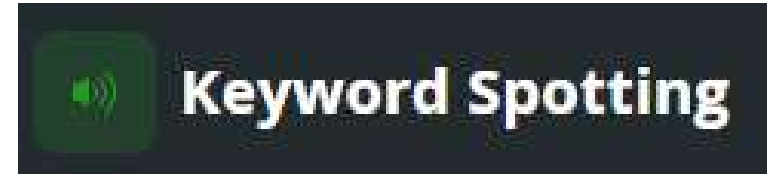
Arduino UNO Q (Ep.35) ถ่าย
ซ่อมคลิป Ep.33



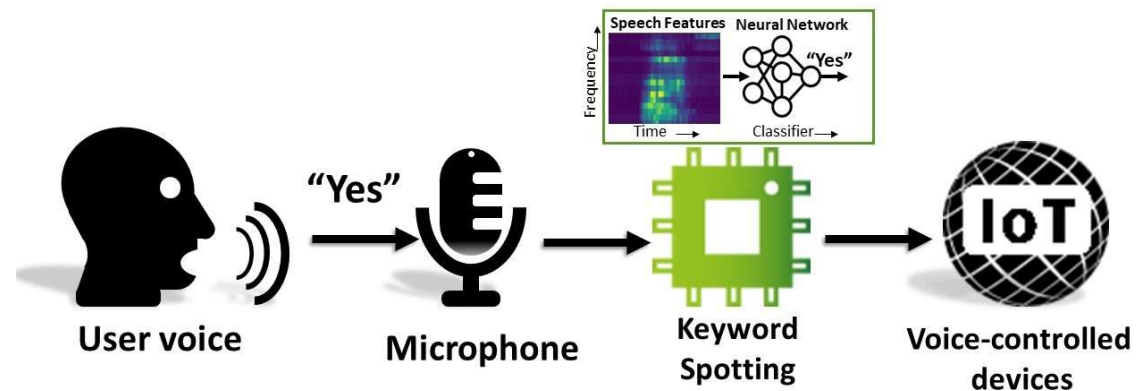
Arduino UNO Q (Ep.33)
กระบวนการ Deploy ไฟล์โมเดล...

Keyword Spotting (KWS)

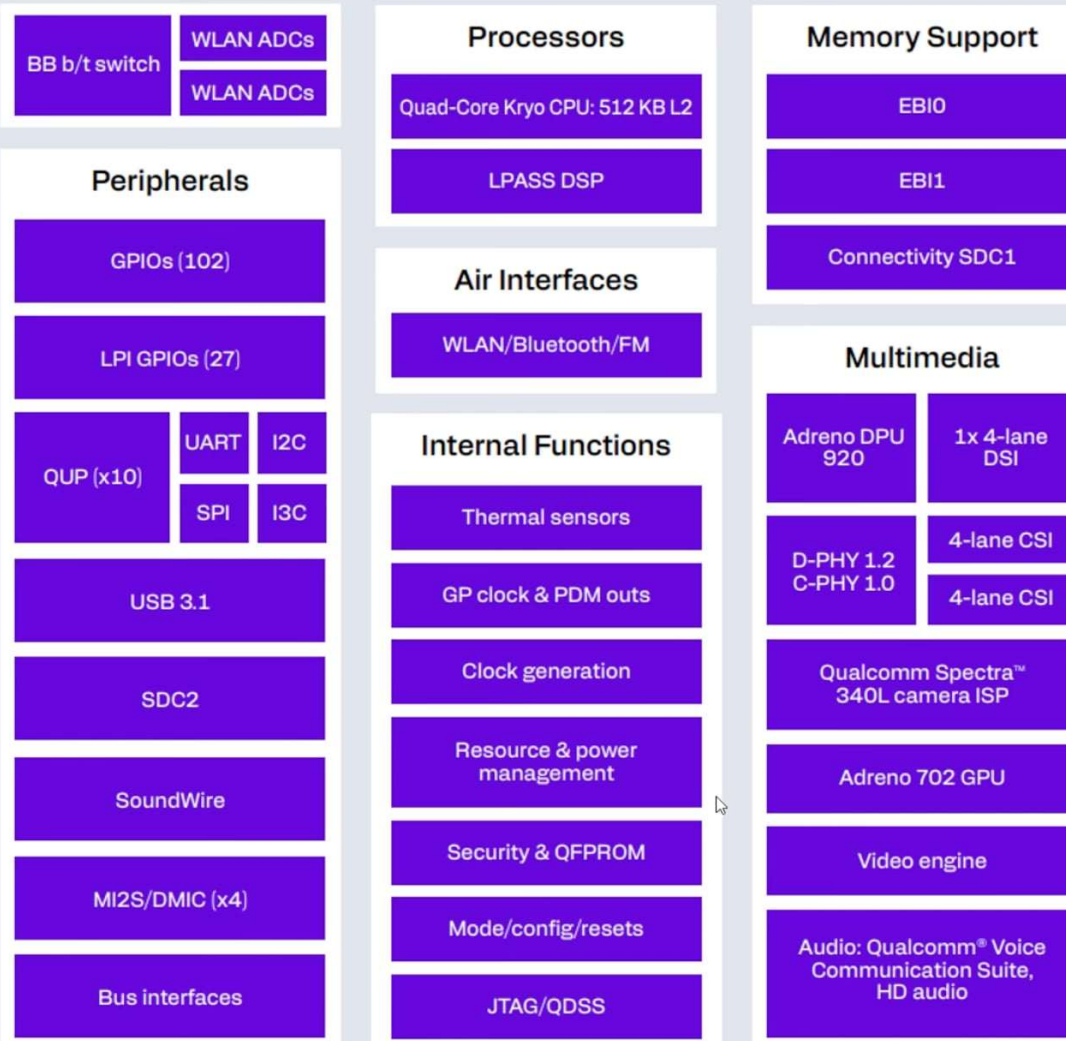
- ระบบตรวจจับคำสั่งเสียงแบบเรียลไทม์
- เทคโนโลยีที่ทำให้อุปกรณ์สามารถจำคำปลุกหรือคำสั่งเฉพาะได้ เช่น “Hey Siri” “OK Google”



Hello Edge : Low Power Keyword Spotting System



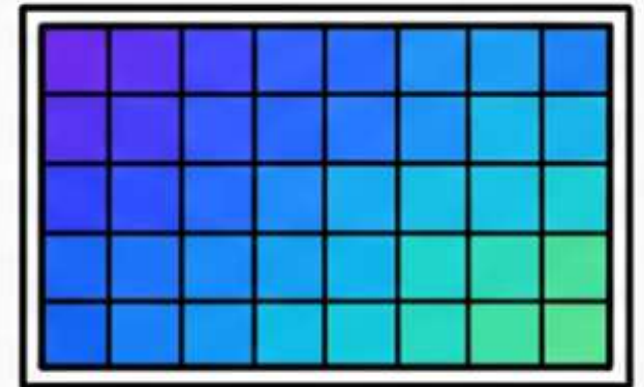
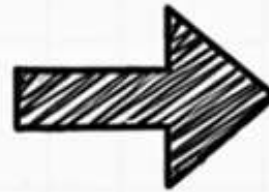
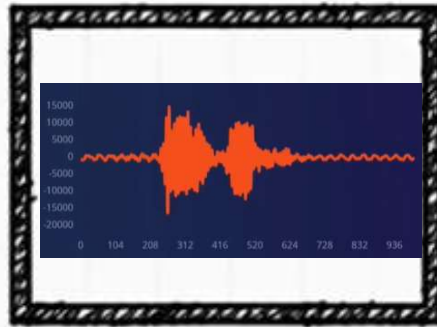
Dragonwing QRB2210



Pre-processing



Spectrogram



keyword



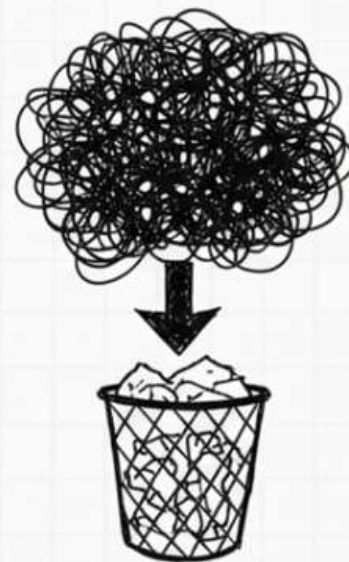
unknown



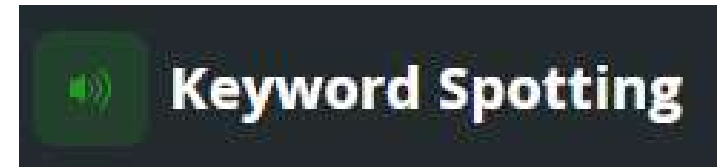
ignore



noise



Keyword Spotting Bridge คืออะไร



- Brick ที่ช่วยให้ Arduino UNO Q ฟังเสียงได้
- รองรับโมเดลสำเร็จรูปและโมเดลจาก Edge Impulse (.eim)
- Trigger event เมื่อพบคำสั่ง
- **วัตถุประสงค์:** Brick ก้อนนี้ถูกออกแบบมาเป็นพิเศษเพื่อ ตรวจจับคำหลัก (Wake Words หรือมีอีกชื่อว่า Keyword) ที่เฉพาะเจาะจงและสั้น เช่น "Alexa", "Hey Google", หรือคำสั่งสั้น ๆ ที่กำหนดเอง
- การเทรนบน Edge Impulse: ในการทำ Keyword Spotting บน Edge Impulse จะใช้การประมวลผลสัญญาณ (DSP) และสถาปัตยกรรมโมเดล (Neural Network) ที่เหมาะสมที่สุด สำหรับงานประเภทนี้ โดยเฉพาะ เพื่อให้มีประสิทธิภาพสูงและใช้พลังงานต่ำ
- เอาต์พุต (Output): จะให้ผลลัพธ์ว่า **คำหลัก (Keyword)** ที่ต้องการถูกตรวจพบหรือไม่ (เช่น 95% เป็นคำว่า "On"), รวมถึงการจำแนกระหว่างคำหลัก, **เสียงรบกวน (Noise)**, และ **เสียงอื่น ๆ ที่ไม่เกี่ยวข้อง (Unknown)**

ส่วนประกอบของการทดลอง

LAVALIER MICROPHONE

WITH EXTENDED AUDIO JACK

M36



CLEAR SOUND
QUALITY



PLUG
AND PLAY



HIGHLY
SENSITIVE



WIDELY
COMPATIBLE

Omni
Directinal

1.5M
CABLE
LENGTH

Microphone : Omni-directional
Signal-to-noise ratio : $\geq 70\text{db}$
Sensitivity : $-38\text{dB}\pm 2\text{Db}$
Frequency range : $100\text{Hz} - 20\text{Khz}$
Cable Length : 1.5 meter
Packing size : $3*8.2*13.7\text{ cm}$
Packing weight : 50 g



8 859790 024350

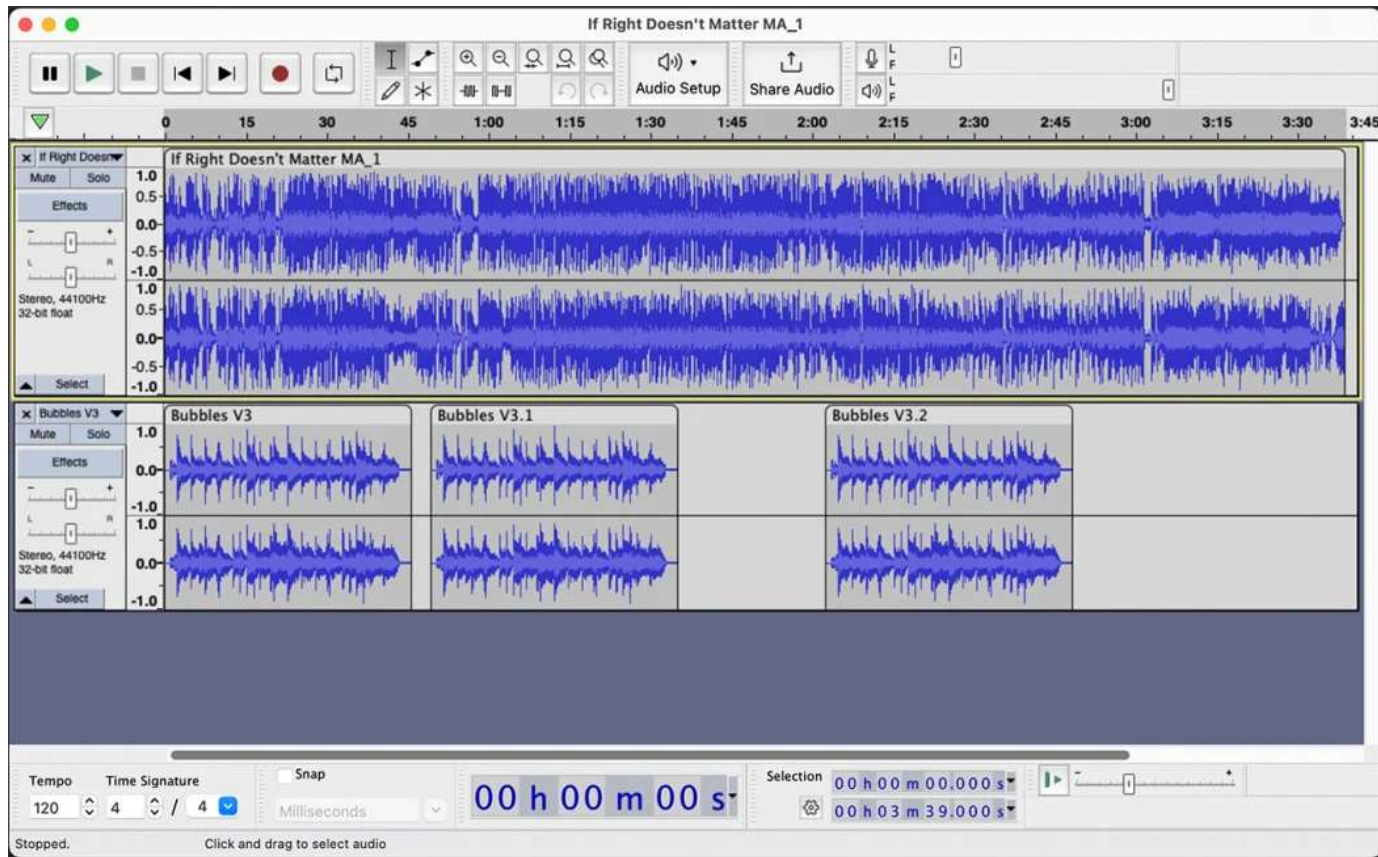


กล้องเว็บแคมก็มีไมค์ในตัว



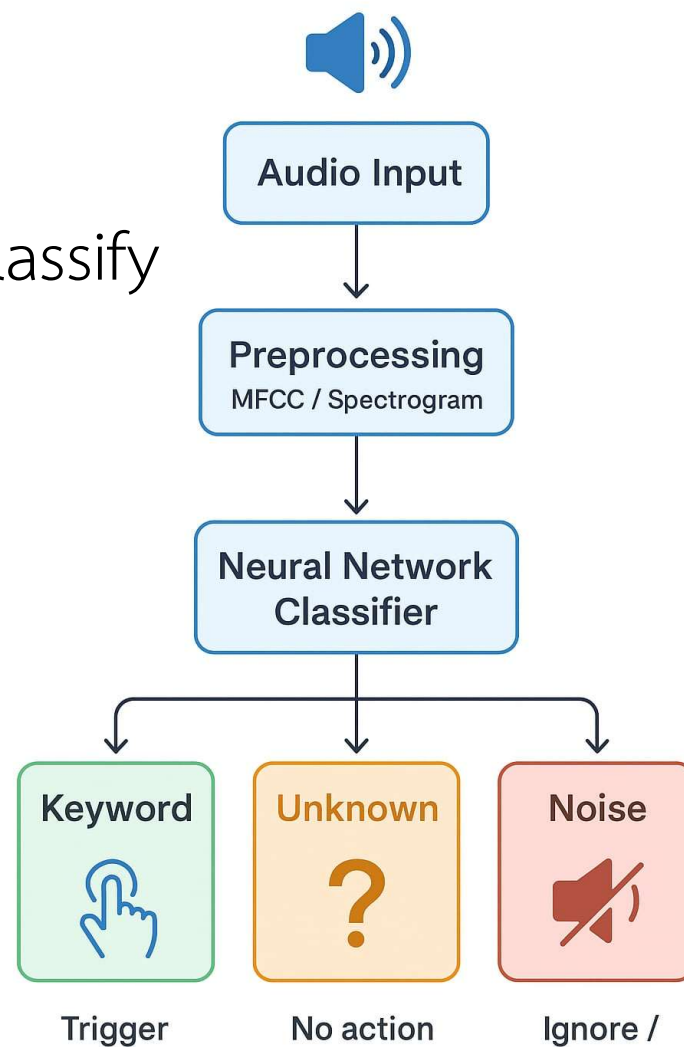
TP-Link_1939



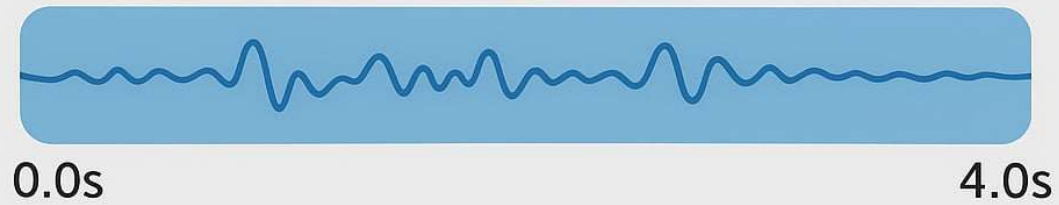


การทำงานของระบบ (Flow)

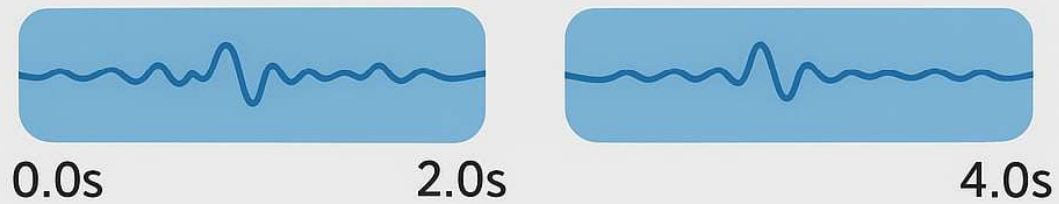
- Audio → Preprocess → AI Model → Classify
- ตรวจสอบความมั่นใจ (Confidence)
- ส่ง event ผ่าน on_detect()



Command



Segment 1-2 seconds



Segment 1 second



คลาส

KeywordSpotting()

KeywordSpotting(confidence=0.9, debounce_sec=3.0)

confidence (float) (optional) ระดับความมั่นใจในการตรวจจับ ตั้งแต่ 0.0 – 1.0 ค่าเริ่มต้นคือ 0.8 (80%) ยิ่งตั้งค่าสูง จะช่วยลด false positive

debounce_sec (float) (optional) ระยะเวลาขั้นต่ำ (วินาที) ระหว่างการตรวจพบคีย์เวิร์ดเดิมซ้ำ ค่าเริ่มต้นคือ 2.0 วินาที

เมธอด

on_detect start() stop()

ระดับ	จำนวนไฟล์แนะนำ	เหมาะกับ
ขั้นต่ำสุด	20–30 ไฟล์	ทดลองสร้างโมเดลเบื้องต้น
ใช้งานได้จริง	50–100 ไฟล์	โมเดลเริ่มมีความแม่นยำดี
คุณภาพสูง	150–300 ไฟล์	โมเดลเสถียรมาก ใช้งานจริงในอุปกรณ์ IoT

Edge Impulse จะแบ่งให้ประมาณ 80 ไฟล์ → สำหรับ Train 20 ไฟล์ → สำหรับ Test ซึ่ง 100 ไฟล์ = จำนวนที่ต้องเตรียมจริง (รวม test แล้ว)

ประเภทคลาส	จำนวนไฟล์ (นามสกุล .wav)	ความยาวรวมขั้นต่ำที่แนะนำ	บทบาทในการสร้างโมเดล
Keyword (เช่น "bed")	50 - 100 ไฟล์	ประมาณ 1 – 3 นาที (ไฟล์ละ ~1 วินาที)	ช่วยให้โมเดลเรียนรู้และจำแนกคำสั่งเฉพาะนี้ได้อย่างแม่นยำ
Noise (เสียงรบกวนพื้นหลัง)	30 - 60 ไฟล์	30 – 90 วินาที	สอนให้โมเดลแยกแยะเสียงรบกวนทั่วไปออกจาก Keyword, ช่วยให้โมเดลมีความทนทานต่อสภาพแวดล้อมจริง
Unknown (คำที่ไม่ใช่ Keyword)	20 – 50 ไฟล์ (จากหลายคำ)	20 – 60 วินาที	สอนให้โมเดลปฏิเสธคำที่ไม่ใช่ Keyword และไม่ใช่เสียงรบกวน, อาจเป็นคำสั่งอื่นที่ไม่ได้ใช้หรือคำที่ฟังคล้ายแต่ผิด

Keyword: ควรมีจำนวนมากพอที่จะครอบคลุมความหลากหลายของการออกเสียง (โทนเสียง, สำเนียง, ระดับเสียง) จากผู้พูดหลาย ๆ คน

Noise: ควรมีมากที่สุดเท่าที่จะทำได้และความหลากหลายสูง

เสียงที่ควรมี:เสียงคงที่ (Ambient Noise): เครื่องปรับอากาศ, พัดลม, ตู้เย็น, เสียงคอมพิวเตอร์

เสียงแบบสุ่ม (Random Noise): เสียงประตูเปิดปิด, เสียงการจราจรเบาๆ, เสียงคนคุยกันในระยะไกล

เป้าหมาย: เพื่อป้องกัน False Positive (โมเดลคิดว่าเสียงรบกวนคือ "bed") และทำให้โมเดลทำงานได้ดีในสภาพแวดล้อมที่มีเสียงดัง

Unknown (คำที่ไม่ใช่ Keyword) จำนวน: ควรมีจำนวนใกล้เคียงกับ Keyword เพื่อไม่ให้โมเดลให้น้ำหนักกับ Keyword มากเกินไปเสียงที่ควรมี:คำสั่งอื่น ๆ: คำสั่งเสียงอื่น ๆ ที่คุณไม่ได้ต้องการให้ระบบตอบสนอง (เช่น "hello", "turn on") คำที่ฟังดูคล้าย: คำที่ออกเสียงคล้าย "bed" เช่น "red", "dead", "said"เสียงที่ไม่มีความหมาย: เสียงบ่นพึมพำ, เสียงหัวเราะ, เสียงไอเป้าหมาย: เพื่อป้องกัน False Positive จากคำที่คล้ายกันหรือคำอื่น ๆ ที่ผู้ใช้พูด

Unknown (คำที่ไม่ใช่ Keyword)

แดง	แดน	แทง	แรง
เปิด	เปิป	เปิด	
เขี้ยว	เคี้ยว	เขี้ยว	เคี้ยว

False Activation Rate (FAR) หรือ False Alarm Rate

FAR คืออัตราที่ระบบให้ผลบวกที่ผิดพลาด (False Positive)

ความหมาย: คืออัตราที่ระบบ ตรวจพบ ว่ามีเหตุการณ์เกิดขึ้น หรือมีการระบุตัวตนที่ถูกต้อง ทั้งที่ความเป็นจริงแล้วไม่ได้เป็นเช่นนั้น (ไม่มีเหตุการณ์ หรือเป็นบุคคลอื่น)

คำจำกัดความทางเทคนิค: $FAR = \text{False Positives} / \text{Total Negative Samples}$

ผลกระทบ:

ถ้า FAR สูง แสดงว่าระบบมีความ ไวเกินไป (Too sensitive) จะมีการแจ้งเตือนที่ไม่จำเป็นบ่อยครั้ง (False Alarms) ซึ่งอาจทำให้ผู้ใช้งานเกิดความรำคาญและไม่เชื่อถือในระบบ

ตัวอย่าง: ในระบบตรวจจับเสียงสั่งงาน (Wake Word Detection) หาก FAR สูง แปลว่าโทรศัพท์ของคุณจะเปิดใช้งานบ่อย ๆ ทั้งที่ไม่มีใครพูดคำสั่งนั้น

วัตถุประสงค์: โดยทั่วไปต้องการให้ค่า FAR ต่ำที่สุด

False Rejection Rate (FRR) หรือ False Non-Match Rate

FRR คืออัตราที่ระบบ ให้ผลลบที่ผิดพลาด (False Negative)

- **ความหมาย:** คืออัตราที่ระบบ ไม่สามารถตรวจพบ เหตุการณ์ที่เกิดขึ้นจริง หรือปฏิเสธบุคคลที่มีสิทธิ์ ทั้งที่ความเป็นจริงแล้วเป็นเช่นนั้น (มีเหตุการณ์จริง หรือเป็นบุคคลที่ถูกต้อง)
- **คำจำกัดความทางเทคนิค:** $FRR = \text{False Negatives} / \text{Total Positive Samples}$
- **ผลกระทบ:**
 - ถ้า FRR สูง แสดงว่าระบบมี **ความเข้มงวดเกินไป** (Too strict) จะพลาดการตรวจจับเหตุการณ์สำคัญหรือปฏิเสธผู้ใช้งานที่ถูกต้อง
 - **ตัวอย่าง:** ในระบบการยืนยันตัวตนด้วยลายนิ้วมือ หาก FRR สูง แปลว่าระบบจะไม่ยอมให้คุณเข้าถึงบ่อย ๆ ถึงแม้ว่าคุณจะใช้ลายนิ้วมือที่ถูกต้องก็ตาม
 - **วัตถุประสงค์:** โดยทั่วไปต้องการให้ค่า FRR ต่ำที่สุด

ความสัมพันธ์และการแลกเปลี่ยน (Trade-off)

โดยปกติแล้ว FAR และ FRR จะมีความสัมพันธ์แบบ ผกผัน กัน:

- ถ้าเพิ่มความเข้มงวด ของเกณฑ์การตรวจจับ (Detection Threshold): จะทำให้ FAR ลดลง (มีการแจ้งเตือนผิดพลาดน้อยลง) แต่จะทำให้ FRR เพิ่มขึ้น (พลาดการตรวจจับเหตุการณ์จริงมากขึ้น)
- ถ้าลดความเข้มงวด ของเกณฑ์การตรวจจับ: จะทำให้ FRR ลดลง (ตรวจจับเหตุการณ์จริงได้มากขึ้น) แต่จะทำให้ FAR เพิ่มขึ้น (แจ้งเตือนผิดพลาดมากขึ้น)

เป้าหมายของการปรับจูน คือ การหาจุดที่สมดุลที่ยอมรับได้ระหว่างสองค่านี้ การเลือกจุดที่เหมาะสมนั้นขึ้นอยู่กับลักษณะการใช้งาน:

- ถ้าความปลอดภัยสำคัญที่สุด (เช่น ระบบเตือนไฟไหม้): ควรเลือก FRR ต่ำ แม้ว่า FAR จะสูงขึ้นเล็กน้อย (ดีกว่าพลาดการเตือนไฟไหม้)
- ถ้าความสะดวกสบายและลดความรำคาญสำคัญที่สุด (เช่น ระบบเปิดไฟอัตโนมัติ): ควรเลือก FAR ต่ำ เพื่อหลีกเลี่ยงการเปิดไฟโดยไม่จำเป็น



Suggested config



Saved config

False rejection rate (FRR)



LABEL	FAR ⑦	FRR ⑦	TRUE POSITIVES ⑦	FALSE POSITIVES ⑦	TRUE NEGATIVES ⑦	FALSE NEGATIVES ⑦
no	0%	5%	19	0	79	1
yes	1.3%	5.3%	18	1	79	1
unknown ⑦	14.7%	0%	10	14	81	0

LABEL	ป้ายกำกับที่โมเดลกำลังประเมิน (ในที่นี้คือ 'no', 'yes', 'unknown')
FAR	False Activation Rate (อัตราการแจ้งเตือนเป็นเท็จ)
FRR	False Rejection Rate (อัตราการปฏิเสธเป็นเท็จ)
TRUE POSITIVES	จำนวนครั้งที่โมเดลทายถูกว่าข้อมูลนั้นเป็นป้ายกำกับนี้ และข้อมูลนั้นเป็นป้ายกำกับนี้จริง ๆ
FALSE POSITIVES	จำนวนครั้งที่โมเดลทายผิดว่าข้อมูลนั้นเป็นป้ายกำกับนี้ ทั้งที่ข้อมูลนั้นไม่ใช่ (เป็นป้ายกำกับอื่น)
TRUE NEGATIVES	จำนวนครั้งที่โมเดลทายถูกว่าข้อมูลนั้น ไม่ใช่ ป้ายกำกับนี้
FALSE NEGATIVES	จำนวนครั้งที่โมเดลทายผิดว่าข้อมูลนั้น ไม่ใช่ ป้ายกำกับนี้ ทั้งที่ข้อมูลนั้นเป็น

LABEL	FAR ⑦	FRR ⑦	TRUE POSITIVES ⑦	FALSE POSITIVES ⑦	TRUE NEGATIVES ⑦	FALSE NEGATIVES ⑦
no	0%	5%	19	0	79	1
yes	1.3%	5.3%	18	1	79	1
unknown ⑦	14.7%	0%	10	14	81	0

LABEL	FAR	FRR	การตีความประสิทธิภาพ
no	0%	5%	ยอดเยี่ยมในการหลีกเลี่ยง False Alarm: FAR เป็น 0% หมายถึง โมเดลไม่เคยทายผิดว่ามี 'no' เกิดขึ้น ทั้งที่ความจริงแล้วไม่ใช่ (FP = 0) อย่างไรก็ตาม มีโอกาสพลาดการตรวจจับ 'no' จริง ๆ อยู่ 5% (FN = 1)
yes	1.3%	5.3%	ดีและสมดุล: มี FAR ต่ำ (1.3%) และ FRR ใกล้เคียงกับ 'no' (5.3%) หมายถึงมีการทายผิดว่าเป็น 'yes' น้อยมาก (FP = 1) และมีการพลาดการตรวจจับ 'yes' จริง ๆ ไป 1 ครั้ง
unknown	14.7%	0%	มีความละเอียดอ่อนสูงแต่ผิดพลาดบ่อย: FRR เป็น 0% หมายถึง โมเดลไม่เคยพลาดการตรวจจับข้อมูลที่ควรเป็น 'unknown' เลย (FN = 0) แต่ มี FAR สูงถึง 14.7% (FP = 14) หมายความว่า โมเดลมีแนวโน้มที่จะทายว่าข้อมูลเป็น 'unknown' บ่อยครั้ง ทั้งที่ความจริงแล้วมันควรจะเป็น 'no' หรือ 'yes'

LABEL	FAR ⑦	FRR ⑦	TRUE POSITIVES ⑦	FALSE POSITIVES ⑦	TRUE NEGATIVES ⑦	FALSE NEGATIVES ⑦
no	0%	5%	19	0	79	1
yes	1.3%	5.3%	18	1	79	1
unknown ⑦	14.7%	0%	10	14	81	0

จุดที่ควรสังเกตเป็นพิเศษ

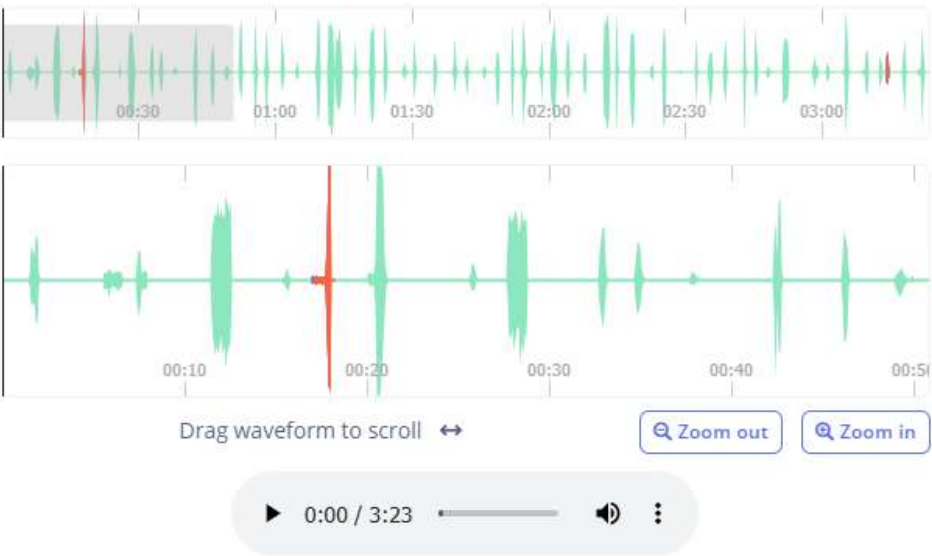
Label 'no': เป็น Label ที่มีความมั่นคงที่สุดในการจำแนก ไม่มีการแจ้งเตือนผิดพลาด (FAR = 0%)

Label 'unknown': เป็น Label ที่มีปัญหาเรื่อง False Positives สูงที่สุด (FP = 14) ซึ่งทำให้ FAR สูงถึง 14.7% สิ่งนี้อาจบ่งชี้ว่า:

- มีข้อมูลที่ไม่ชัดเจน/กำกวมจำนวนมากในชุดข้อมูล
- หรือเกณฑ์การตั้งค่า (Threshold) สำหรับ 'unknown' อาจจะกว้างเกินไป ทำให้โมเดลจัดให้ข้อมูลที่ไม่แน่ใจว่าเป็น 'unknown' มากเกินไป
- FRR เป็น 0% ก็เป็นสัญญาณที่ดีในแง่ที่มันไม่เคยพลาดการตรวจจับ 'unknown' เลย

Results for selected config

Shows any errors your impulse makes on a sample of data.



ERROR	TYPE	LABEL	START TIME	
False negative		no	16.933s	Play
False positive	Incorrect match	no → noise (sample)	17s	Play
False positive	Incorrect match	no → noise (sample)	17.25s	Play
False negative		yes	3m 13.74s	Play
False positive	Incorrect match	noise → yes	3m 13.75s	Play

ตารางแสดงรายการข้อผิดพลาดแต่ละรายการ โดยระบุ:

ERROR (ประเภทข้อผิดพลาด):

- False negative (ผลลบที่เป็นเท็จ): โมเดล พลาด การตรวจจับเหตุการณ์ที่เกิดขึ้นจริง
- False positive (ผลบวกที่เป็นเท็จ): โมเดล ตรวจจับ เหตุการณ์ที่ไม่เกิดขึ้นจริง (แจ้งเตือนผิด)

TYPE (ชนิด): ระบุประเภทของความผิดพลาด เช่น

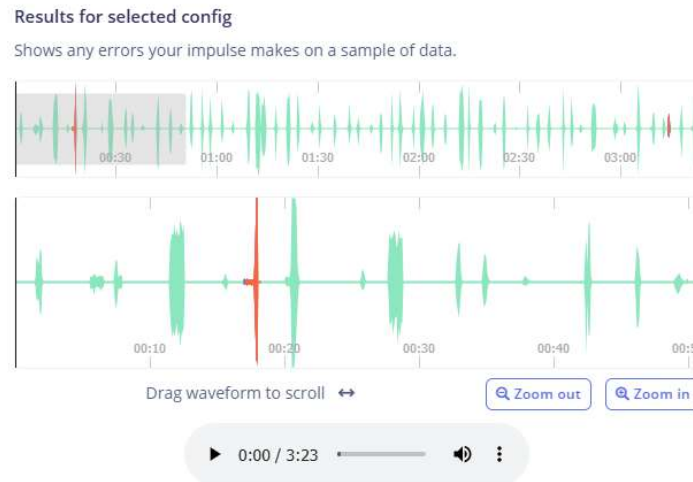
- Incorrect match: การจับคู่ผิดพลาด (สำหรับ False positive)

LABEL (ป้ายกำกับ):

- สำหรับ False negative: คือป้ายกำกับที่โมเดลควรจะตรวจจับได้แต่พลาดไป (เช่น ควรเป็น 'no' แต่โมเดลไม่ได้ทายอะไร)
- สำหรับ False positive: คือข้อมูลที่โมเดลทายผิดว่าเป็นป้ายกำกับนั้น (แสดงเป็น \$Original \rightarrow Guess\$ เช่น \$no \rightarrow noise\$)

START TIME (เวลาเริ่มต้น): ตำแหน่งเวลาที่แน่นอนในไฟล์เสียงที่เกิดข้อผิดพลาดขึ้น

Play: ปุ่มสำหรับเล่นเสียงในช่วงเวลาที่เกิดข้อผิดพลาด เพื่อให้ผู้ใช้สามารถฟังและตรวจสอบข้อผิดพลาดด้วยตัวเองได้



รูปคลื่นเสียง (Waveform) ของไฟล์ข้อมูลทั้งหมด (3:23 นาที) โดยมีแถบที่เน้นช่วงเวลาที่เกิดข้อผิดพลาด:

สีแดง: มักจะหมายถึง False positive (การแจ้งเตือนผิดพลาด)

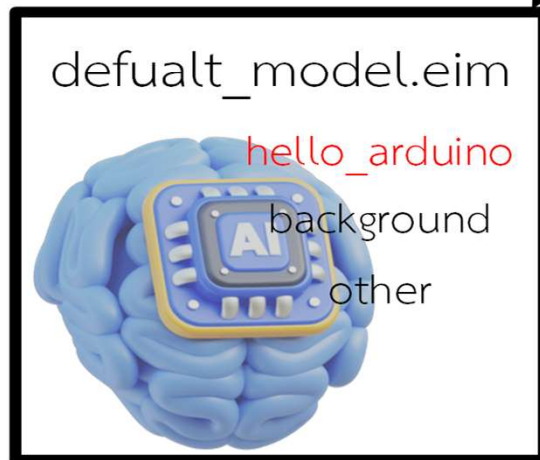
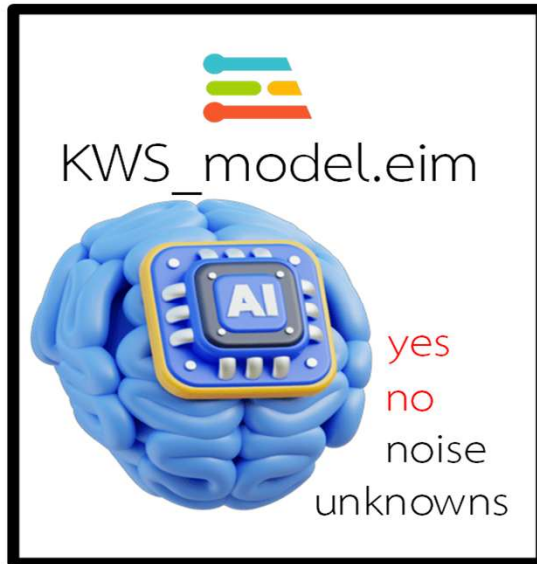
สีน้ำเงิน: มักจะหมายถึง False negative (การพลาดการตรวจจับ)

การดูรูปคลื่นเสียงพร้อมกับรายการข้อผิดพลาด ช่วยให้วิศวกรเข้าใจว่า สัญญาณเสียง ที่ทำให้เกิดข้อผิดพลาดมีลักษณะอย่างไร เพื่อนำไปปรับปรุงการตั้งค่าพารามิเตอร์ (เช่น Averaging window duration หรือ Detection threshold) ให้เหมาะสมยิ่งขึ้น

ERROR	TYPE	LABEL	START TIME	
False negative		no	16.933s	Play
False positive	Incorrect match	no → noise (sample)	17s	Play
False positive	Incorrect match	no → noise (sample)	17.25s	Play
False negative		yes	3m 13.74s	Play
False positive	Incorrect match	noise → yes	3m 13.75s	Play

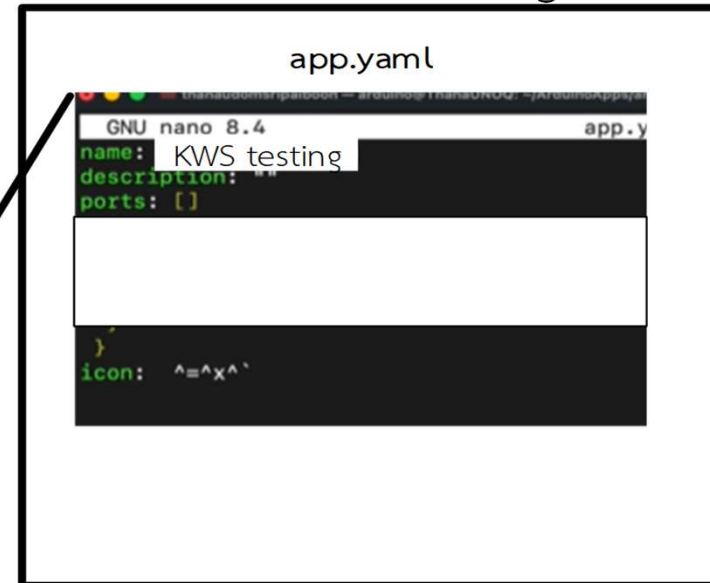
ประเภทข้อผิดพลาด	LABEL	Start Time	การตีความ
False negative	no	16.933s	โมเดล พลาด การตรวจจับ 'no' ที่เกิดขึ้นจริง
False positive	no → noise	17s	โมเดล ทายผิด ว่าเสียง 'no' จริง ๆ เป็นเสียง 'noise'
False positive	no → noise	17.25s	โมเดล ทายผิด ว่าเสียง 'no' จริง ๆ เป็นเสียง 'noise' (ข้อผิดพลาดลักษณะเดิมที่เกิดขึ้นซ้ำ)
False negative	yes	3m 13.74s	โมเดล พลาด การตรวจจับ 'yes' ที่เกิดขึ้นจริง
False positive	noise → yes	3m 13.75s	โมเดล ทายผิด ว่าเสียง 'noise' จริง ๆ เป็นเสียง 'yes' (เกิดหลัง False negative 'yes' เกือบจะทันที)

/home/arduino/.arduino-bricks/ei-models/

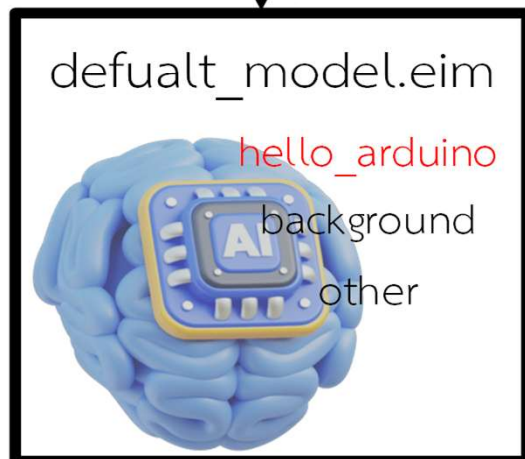
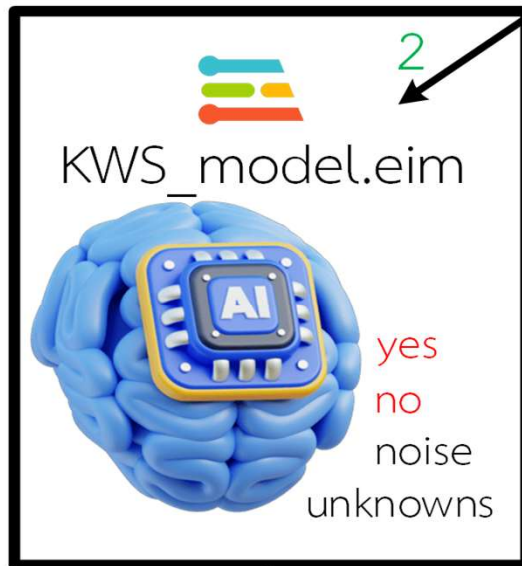


1

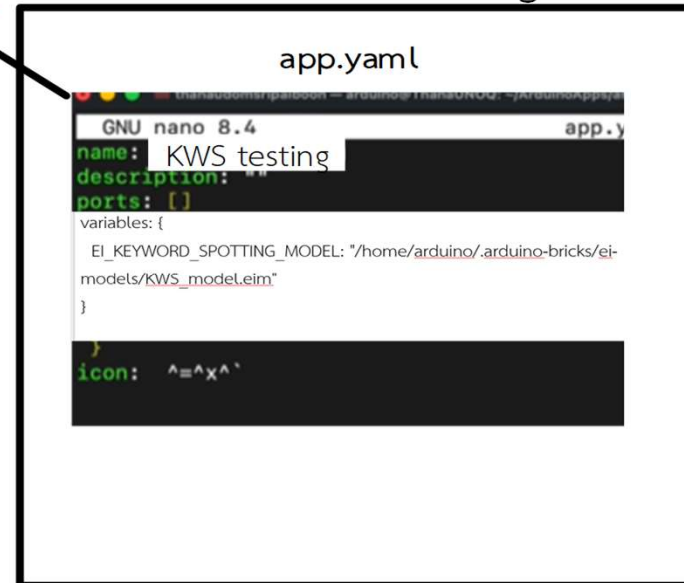
KWS testing



/home/arduino/.arduino-bricks/ei-models/



KWS testing



ขั้นตอนการ Deploy (โหมด Network)

1. สร้าง App ขึ้นมารองรับโดยตั้งชื่อว่า KWS testing
2. ปิด App เข้า Terminal พร้อมที่จะทำการ Access บน Command Line บนลินุกซ์
3. เตรียมไฟล์ .eim จากแฟลชไดรฟ์ เสียบเข้า Dongle ที่ต่อเข้ากับ Arduino UNO Q
4. Terminal เข้าตัวบอร์ด Arduino UNO Q โดยโหมด Network Mode
5. ที่หน้าจอรูเริ่มด้วยคำสั่งตรวจสอบและ mount ตัว Flash Drive ดังนี้

lsblk

```
sudo blkid /dev/sda1
```

```
sudo mkdir -p /media/usb
```

```
sudo mount /dev/sda1 /media/usb
```

```
df -h
```

ขั้นตอนการ Deploy (โหมด Network)

6. ทำการตรวจสอบว่ามีห้อง ei-models หรือยังโดยใช้คำสั่ง

```
ls -l /home/arduino/.arduino-bricks/ei-models/
```

ถ้ายังใช้คำสั่ง

```
mkdir -p /home/arduino/.arduino-bricks/ei-models/
```

7. ทำการคัดลอกไฟล์ .eim ที่ได้ทำการเทรนมาบน Edge Impulse

```
cp /media/usb/KWS_model.eim /home/arduino/.arduino-bricks/ei-models/
```

8. ตรวจสอบว่ามีไฟล์อยู่ในโฟลเดอร์หรือไม่

```
ls -l /home/arduino/.arduino-bricks/ei-models/
```

9. เปิดไฟล์ app.yaml ของ App ขึ้นมา

```
nano /home/arduino/ArduinoApps/kws-testing/app.yaml
```

ขั้นตอนการ Deploy (โหมด Network)

10. เพิ่มหรือแก้ไขไฟล์ app.yaml ใน App

```
variables: {
```

```
  EI_KEYWORD_SPOTTING_MODEL: "/home/arduino/.arduino-bricks/ei-  
models/KWS_model.eim"  
}
```

11. หลังจากแก้ไขแล้ว: บันทึกไฟล์ (ใน nano: Ctrl+O แล้ว Enter, จากนั้น Ctrl+X เพื่อออก)

12. ถอดยูเอสบีออก

```
sudo umount /media/usb
```

คำสั่ง `get_model_info()` มักจะถูกใช้ในคลาส Keyword Spotting (หรือคลาสอื่น ๆ ใน Arduino App Lab ที่ใช้ไฟล์ .eim) เพื่อ ดึงข้อมูล metadata ของโมเดล AI ที่คุณโหลดเข้ามา เช่น ชื่อโมเดล, เวอร์ชัน, อินพุต-เอาต์พุต, รายละเอียดคลาสต่าง ๆ เป็นต้น

คำสั่ง `dir()` เป็นคำสั่งพื้นฐานใน Python ที่ใช้เพื่อ ดูรายชื่อ attribute และ method ทั้งหมดที่ออบเจกต์นั้นมีอยู่

คำสั่ง `getattr()` เป็นฟังก์ชันพื้นฐานของ Python ที่ใช้เพื่อ ดึงค่า attribute ของออบเจกต์ โดยใช้ชื่อเป็นสตริง เหมาะมากเมื่อคุณต้องการเข้าถึง attribute แบบไม่รู้ชื่อมาก่อน (เช่นในลูป)

คำสั่ง `callable()` ใน Python ใช้เพื่อตรวจสอบว่า ค่าวัตถุ (object) ที่กำลังตรวจสอบอยู่นั้น “เรียกใช้งานได้เหมือนฟังก์ชัน” หรือไม่หรือพูดง่ายๆ คือ “เป็นฟังก์ชันหรือเป็นวัตถุที่เรียก (call) ได้หรือเปล่า

คำสั่ง `startswith()` เป็นเมธอดของ string ใน Python ที่ใช้เพื่อ ตรวจสอบว่าข้อความ (string) เริ่มต้นด้วยตัวอักษรหรือคำที่กำหนดหรือไม่

```
from arduino.app_bricks.keyword_spotting import KeywordSpotting
from arduino.app_utils import *
```

```
spotter = KeywordSpotting()
```

```
def print_kws_model_info(spotter):
```

```
    info = spotter.get_model_info()
```

```
    if info is None:
```

```
        print("ยัง無法เข้าถึงข้อมูลจากโมเดลได้")
```

```
        return
```

```
print("=== ข้อมูลของคลาสที่อยู่ในตัว AI โมเดล ===")
```

```
for attr in dir(info):
```

```
    # กรอง method และ private attribute
```

```
    if not attr.startswith("_") and not callable(getattr(info, attr)):
```

```
        print(f"{attr}: {getattr(info, attr)}")
```

```
from arduino.app_bricks.keyword_spotting import KeywordSpotting
from arduino.app_utils import App
spotter = KeywordSpotting()
spotter.on_detect("helloworld", lambda: print(f"Hello world detected!"))
App.run()
```

```
from arduino.app_bricks.keyword_spotting import KeywordSpotting
from arduino.app_utils import App
import time
```

```
spotter = KeywordSpotting()
```

```
time.sleep(1) # รอโมเดลโหลด
```

```
info = spotter.get_model_info()
```

```
if info is None:
```

```
    print("ยังไม่สามารถดึงข้อมูลโมเดลได้")
```

```
else:
```

```
    print("=== Classes ในโมเดล ===")
```

```
    for label in info.labels:
```

```
        print("-", label)
```

```
    # สร้าง callback แบบ function ปกติไม่มี argument
```

```
    def make_callback(l):
```

```
        def callback():
```

```
            print(f"[Detected] {l}")
```

```
        return callback
```

```
    spotter.on_detect(label, make_callback(label))
```

```
App.run()
```